

SkillMatch AI: Delivery, Evaluation, and Professional Development

James Lambert

System Delivery and Integration

SkillMatch addresses a workforce planning problem: job titles do not adequately describe whether employees possess the skills, valid certifications, and experience needed for specific work. My implementation combines structured profiles with job requirements to produce ranked, explainable recommendations while leaving staffing authority with a supervisor. The integrated workflow connects React, FastAPI, repositories, PostgreSQL, a deterministic matching package, and stored feedback.

Although this was assigned to three people, I completed the implementation and integration work individually. One assigned teammate did not respond, and another did not contribute work. This transferred the planned team workload to me and removed opportunities for independent implementation review. I adapted by prioritizing the complete recommendation path, maintaining explicit contracts, and using automated checks. I used AI assistance for development and review support, but that assistance does not constitute independent peer review.

The computational approach separates eligibility from suitability. Non-ACTIVE employees are ineligible, and missing a valid mandatory certification prevents eligibility. Missing required skills reduces the score without hard exclusion. Required skills, preferred skills, certifications,

and experience contribute through the R/P/C/E model. Deterministic ordering uses final score, required-skill coverage, certification coverage, and employee identifier, making identical input snapshots and model versions reproducible.

Concrete integration work included aligning employee and job endpoints with canonical Pydantic contracts and connecting those contracts to React profile forms. The recommendation service in `backend/src/skillmatch/features/recommendations/service.py` records a snapshot hash and model version, invokes matching, and persists results before returning them. Profile updates and permanent deletions include the last-read version, preventing stale writes from silently replacing newer information. The frontend preserves drafts when `STALE_VERSION` occurs and offers an explicit reload.

These changes made contract consistency a concrete integration priority.

Development proceeded through focused increments covering authentication, role enforcement, profile maintenance, matching, persistence, feedback, and user-facing errors. Resolving merge conflicts required checking the integrated behavior again rather than assuming previously passing files remained compatible. Three PostgreSQL browser scenarios subsequently demonstrated recommendations and feedback plus employee and job lifecycle operations.

The AI feature adds inspectable comparisons rather than unsupported autonomous judgment. Each result provides component scores, evidence gaps, eligibility, and explanation. `SELECTED`, `NOT_SELECTED`, and `DEFERRED` feedback is tied to a stored run without automatic assignment or live retraining. This supports accountability, but demonstration scores do not establish staffing accuracy. Hosted deployment, independent review, and curated relevance evaluation remain outstanding evidence requirements. Delivery therefore demonstrates a functioning local prototype with documented limitations, rather than a completed production release.

Methodology and Performance Evaluation

The feature-oriented modular monolith was appropriate for this prototype because it kept deployment simple while preserving meaningful internal boundaries. Authentication, employee profiles, jobs, recommendations, and feedback remain distinct capabilities. The matcher operates without HTTP or database access; recommendation orchestration owns repository interaction and persistence. This arrangement reduces coupling and makes business behavior independently testable, although a single process limits independent scaling and failure isolation.

The Software Engineering Institute’s Architecture Tradeoff Analysis Method evaluates architecture against interacting quality attributes rather than assuming one design is universally superior (Kazman et al., 1998). My design reflects that reasoning through explicit tradeoffs among maintainability, performance, and operational simplicity. However, I did not conduct a formal ATAM evaluation with stakeholders. A future review should examine concrete scenarios involving concurrent recommendation requests, database outages, and changing certification rules before deciding whether service extraction is justified.

Security practices also align with selected recommendations from NIST’s Secure Software Development Framework, which integrates security into development activities (Souppaya et al., 2022). Bearer authentication, role checks, generic credential failures, schema validation, and versioned updates address identifiable risks. These controls do not establish full framework compliance. Deployment hardening, operational monitoring, and independent security review require additional evidence.

Observed local results provide quantitative support. The backend suite passed 382 tests, with statement coverage of 97.21% and branch coverage of 89.23%. The frontend passed 91 tests across seven files, with line coverage of 94.89% and branch coverage of 81.81%. Three browser scenarios passed against PostgreSQL, and a separate database session verified persisted runs, candidates, and feedback. Migration checking found no schema drift. These results are

recorded in docs/evidence/unit8/repository-evidence.md.

Coverage has practical limits: backend API tests largely use SQLite, and frontend unit tests mock HTTP. The PostgreSQL browser scenarios address integration risk but do not exhaustively cover deployment conditions. Six backend warnings also remained. Test counts and coverage percentages cannot establish absence of defects, production availability, or recommendation relevance.

The benchmark evaluated 100 ACTIVE profiles over 30 sequential requests after five excluded warm-ups. P95 latency was 39.02 milliseconds against the target of less than two seconds. Every measured run persisted 100 candidate results. Timing included authentication, PostgreSQL reads, matching, transaction commit, and response decoding. However, the synthetic, single-job, local warm-cache workload does not demonstrate concurrent capacity or general production scalability.

The strongest reliability decisions were atomic match persistence, typed failure responses, request identifiers, deterministic ranking, and optimistic concurrency. The weakest process area was independent review, because absent teammate participation left review responsibility concentrated with me. Automated tools improved consistency but could not supply independent judgment. The hosted frontend summary showed 91 passing tests, while complete final-commit CI and deployment evidence remained unverified. These remaining gaps should remain clearly visible in the final portfolio assessment.

Professional Development Roadmap

My professional development direction is full-stack engineering with application security as a central responsibility. SkillMatch showed that feature delivery depends on agreement between interfaces, authorization, persistence, and user experience. My next goal is to improve those connections while learning to operate applications beyond a local development environment. I will judge progress through demonstrable changes and recorded outcomes rather than

certificates alone.

During the first three months, I plan to strengthen TypeScript, React, and API contract design. GitHub’s Octoverse 2025 report identifies TypeScript as its most-used language by monthly contributors in August 2025 (GitHub, 2025). That trend supports investigating typed frontend development, although popularity does not prove suitability for every project. I will migrate one bounded SkillMatch workflow to TypeScript, evaluate OpenAPI-based client generation, and test whether incorrect response assumptions are caught before runtime. Success will require preserved browser behavior and a documented comparison of maintenance effort.

My second priority is systematic application security. OWASP’s Application Security Verification Standard provides testable requirements for web application controls (OWASP Foundation, 2025). I will use its stable version 5.0.0 to build a scoped verification checklist covering authentication, authorization, validation, and sensitive data handling. Within six months, I plan to create a threat model, test cross-user access to stored runs, evaluate token storage risks, and practice dependency and secret scanning. I will record findings, severity, remediation, and retest results instead of claiming compliance from a checklist alone.

I also want to use AI-assisted development with stronger verification. Stack Overflow’s 2025 survey reports that 84% of respondents use or plan to use AI tools, while 46% distrust their accuracy (Stack Overflow, 2025). These findings support learning the tools while retaining responsibility for their output. I will compare assisted and unassisted implementation of small tasks using defect rate, review effort, and reproducibility. Generated changes must pass contract, security, and regression checks; confidential data will stay outside unapproved tooling.

Between months six and twelve, I plan to deploy a demonstration application with TLS, managed PostgreSQL, controlled secrets, and repeatable migrations. I will add observable request tracing and concurrent load tests, then document recovery from deployment and

database failures. This addresses the gap between my current local benchmark and reliable operation under realistic conditions.

To maintain adaptability, I will review authoritative survey results, standards, and framework documentation quarterly, maintain a learning backlog, and seek independent code review. A monthly technical note will explain one adoption decision, including alternatives, risks, measured results, and rollback criteria. This process will help me distinguish useful technologies from trends and turn continued learning into verifiable engineering improvement. Security, maintainability, accessibility, and human oversight will remain evaluation criteria as tools and architectures evolve. I will revisit milestones quarterly and adjust them using documented reviewer feedback.

References

- GitHub. (2025). *Octoverse: A new developer joins GitHub every second as AI leads TypeScript to #1*. <https://github.blog/news-insights/octoverse/octoverse-a-new-developer-joins-github-every-second-as-ai-leads-typescript-to-1/>
- Kazman, R., Klein, M. H., Barbacci, M. R., Longstaff, T. A., Lipson, H. F., & Carriere, J. (1998). *The architecture tradeoff analysis method* (CMU/SEI-98-TR-008). Software Engineering Institute, Carnegie Mellon University. <https://www.sei.cmu.edu/library/the-architecture-tradeoff-analysis-method/>
- OWASP Foundation. (2025). *Application Security Verification Standard* (Version 5.0.0). <https://github.com/OWASP/ASVS/releases/tag/v5.0.0>
- Souppaya, M., Scarfone, K., & Dodson, D. (2022). *Secure Software Development Framework (SSDF) version 1.1: Recommendations for mitigating the risk of software vulnerabilities* (NIST SP 800-218). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-218>
- Stack Overflow. (2025). *2025 developer survey: AI*. <https://survey.stackoverflow.co/2025/ai/>